

Monitoring Dashboards for AI Infrastructure

See **system, application, and model** health at a glance with purpose-built dashboards that drive action, not vanity metrics.

Using industry-standard tools: Prometheus, Grafana, NVIDIA DCGM, ELK/EFK, and OpenTelemetry to create SLO-driven views that matter.



What "Good" Monitoring Looks Like



Persona-Focused Views

One dedicated page per role (SRE, ML Engineer, Product)

SLO-Driven Signals

Clear Red/Amber/Green indicators against defined objectives

Connected Context

Panels link directly to relevant logs & traces for fast troubleshooting

Clear Accountability

Every alert has defined ownership and linked runbooks

Observability Stack Reference Architecture

1

Metrics Collection

Prometheus with node-exporter and kube-state-metrics for system and application telemetry

2

GPU Monitoring

NVIDIA DCGM Exporter capturing detailed GPU utilization, memory, power, and thermal metrics

3

Log Aggregation

Fluent Bit forwarding to Elasticsearch/OpenSearch with Kibana for visualization and search

4

Distributed Tracing

OpenTelemetry instrumentation feeding into Tempo or Jaeger for end-to-end request visibility

5

Visualization & Alerting

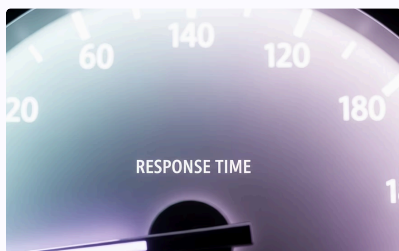
Grafana dashboards with Alertmanager for notification routing and incident management

Critical SLOs to Track from Day One

99.9%

Availability

Monthly uptime target



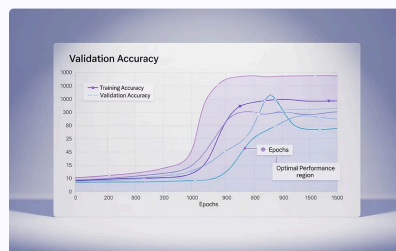
Throughput

QPS / jobs per hour to track system capacity

300ms

p95 Latency

95% of requests faster than this



Model Health

Accuracy proxy metrics and drift detection

0.1%

Error Rate

Maximum acceptable 5xx errors



Cost Efficiency

\$/1k requests, GPU utilization %, idle time

Core Dashboards You Need

Five essential views that cover your entire AI infrastructure stack:

1	API Inference Owner: Platform Team End-user experience metrics including request rates, latencies, and error patterns
2	GPU/Node Health Owner: SRE Team Hardware utilization and health signals across compute resources
3	Model Quality & Drift Owner: ML Engineering Production model performance and statistical drift indicators
4	Data Freshness / Pipeline Owner: Data Engineering Data flow health and feature freshness monitoring
5	Cost & Efficiency Owner: FinOps/Engineering Management Resource utilization and cost attribution across teams



API Inference Dashboard Anatomy

Key Panels

- Requests/sec, **p95 latency**, error rate (4xx/5xx)
- Saturation metrics: CPU, memory, pod restarts
- HPA activity correlated with traffic patterns
- **Top endpoints by latency & region heatmap**

Example PromQL Queries

```
# Request rate (successful)
sum(rate(http_requests_total{
  job="inference",status="2.."
}[5m]))

# p95 latency
histogram_quantile(0.95, sum by (le) (
  rate(http_request_duration_seconds_bucket{
    job="inference"
  }[5m])
))

# Error rate
sum(rate(http_requests_total{
  job="inference",status="5.."
}[5m])) /
sum(rate(http_requests_total{
  job="inference"
}[5m]))
```

GPU/Node Health Dashboard



Critical GPU Metrics to Track

Utilization

GPU core utilization %, memory utilization %

Power & Thermal

Power consumption (W), temperature (°C)

Scheduling

Unschedulable pods, GPU queue length

DCGM PromQL Examples

```
# GPU utilization by device  
avg(nvidia_gpu_duty_cycle) by (gpu)
```

```
# Memory utilization percentage  
avg(nvidia_gpu_memory_used_bytes /  
nvidia_gpu_memory_total_bytes) by (gpu)
```

```
# Power consumption  
avg(nvidia_gpu_power_usage_watts) by (gpu)
```

Model Quality & Drift Dashboard



Online Quality Proxy

A/B testing feedback, win-rate vs baseline model



Prediction Distribution Drift

PSI/JS divergence metrics tracking statistical shift



Feature Drift Detection

Missingness rates, mean/median shifts in distributions



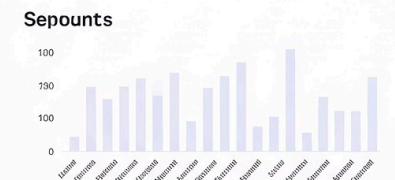
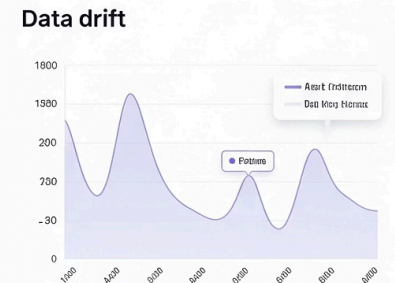
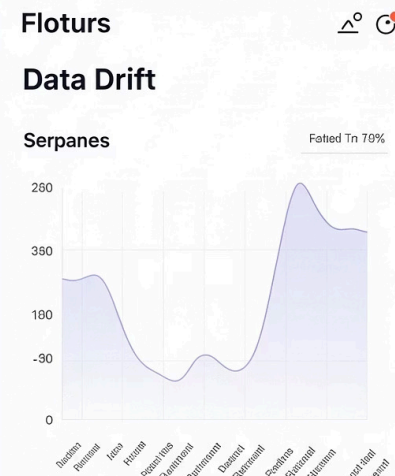
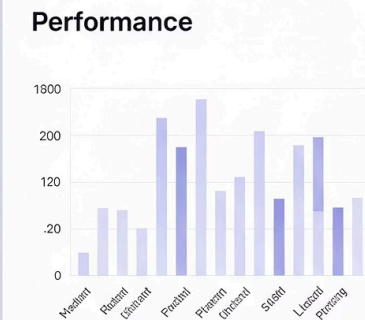
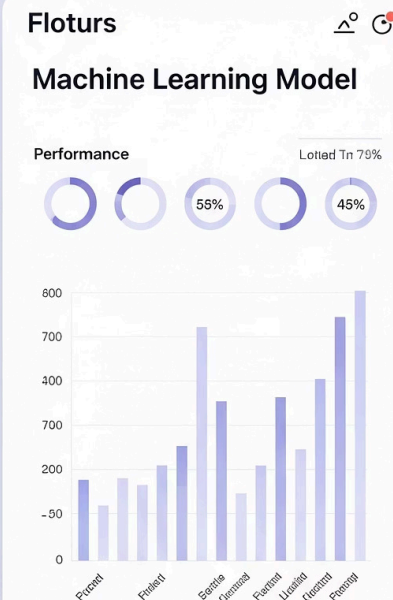
Version Tracking

Model vX.Y release markers overlaid on metrics

Drift Proxy PromQL (custom exporter):

```
avg_over_time(model_js_divergence_score[1h])
```

Link to Evidently/WhyLabs reports for deeper analysis



Data Freshness / Pipeline Dashboard

Key Components to Monitor

Kafka Lag

Per consumer group offset lag tracking

Airflow DAG Status

Success/fail counts, run duration tracking

Data Promotion Timing

Bronze→Silver→Gold promotion latency

Feature Store Freshness

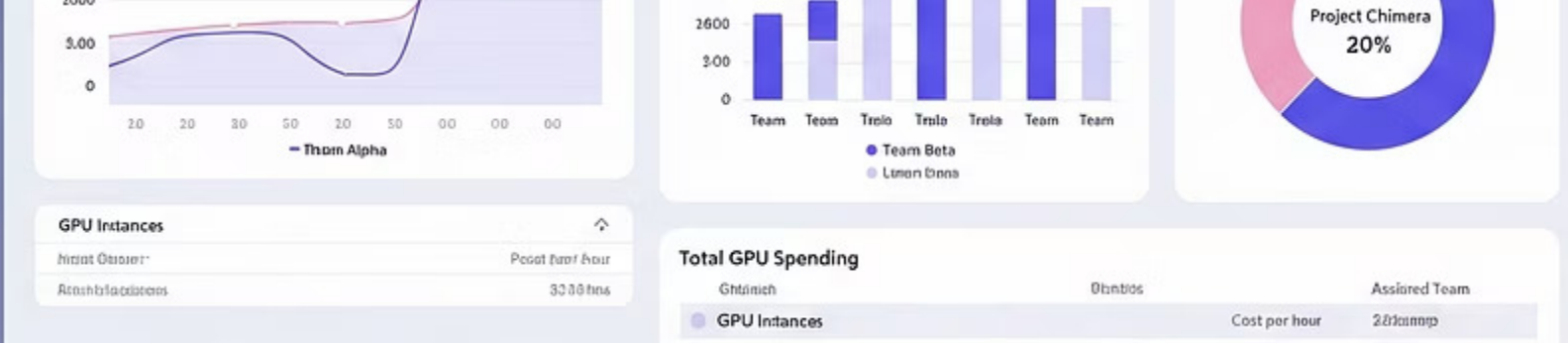
Materialization staleness by feature group



PromQL Examples

```
# Max consumer group lag
max(kafka_consumergroup_lag{
  group="features-stream"
})
```

```
# Average DAG duration
rate(airflow_dag_run_duration_seconds_sum{
  dag_id="build_features"
}[15m]) /
rate(airflow_dag_run_duration_seconds_count{
  dag_id="build_features"
}[15m])
```



Cost & Efficiency Dashboard

Cost Visibility Metrics

- GPU hours by team/environment (via Kubernetes labels)
- **\$/1k requests** cost estimates by service
- Idle GPU time (% under utilization threshold)
- Storage growth (TB) by tier (hot/warm/cold)
- Network egress GB per service/region

Cost Tracking PromQL

```
# Cost per request (needs custom exporter)
sum(rate(inference_requests[5m]))
* on(service) group_left
avg(avg_cost_per_request_usd)

# Idle GPU time percentage
sum(nvidia_gpu_duty_cycle < 10) /
count(nvidia_gpu_duty_cycle) * 100
```

Pro tip: Tag resources by team, project, and environment to enable accurate cost attribution

Health Probes & Golden Signals

Latency

How long requests take to process

- p50/p95/p99 percentiles
- By endpoint/region

Saturation

How "full" your system is

- Resource utilization
- Queue depth

Traffic

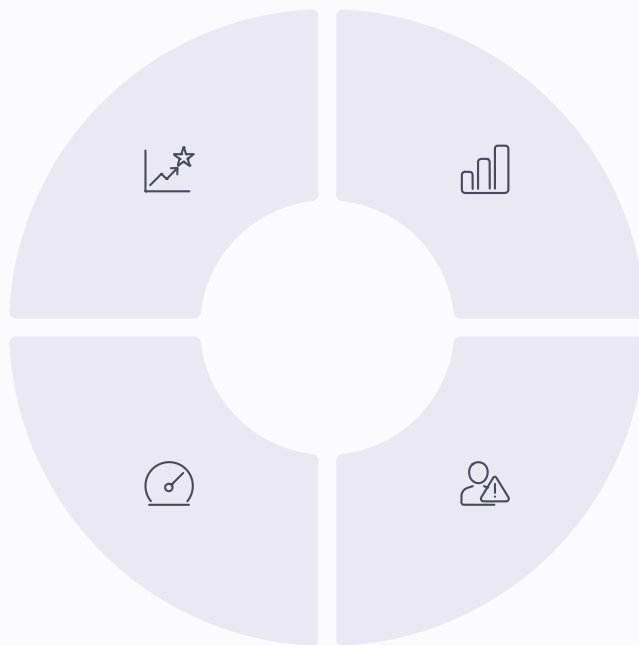
Volume of demand on system

- Requests per second
- Batch jobs per hour

Errors

Failed request rate

- HTTP 5xx rate
- Application exceptions



Burn Rate SLO Panel (30m window)

```
sum(rate(http_requests_total{status=~"5.."}[30m])) /  
sum(rate(http_requests_total[30m]))
```

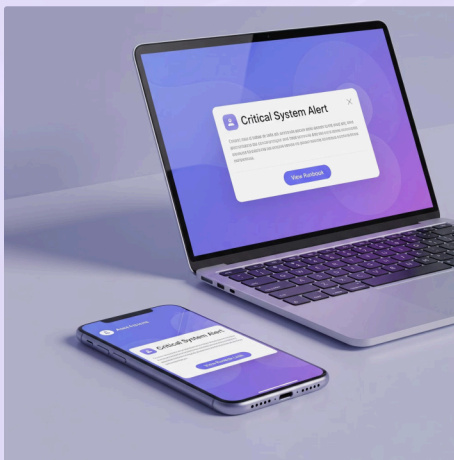
Alerts That Actually Help

Page-Worthy Alerts (Immediate Action)

- p95 latency > SLO for 10m
- 5xx error rate > 1% for 10m
- No ready pods for critical service
- GPU memory utilization > 95%

Ticket-Worthy Alerts (Planned Action)

- Kafka lag > threshold for 30m
- Storage growth anomaly detection
- Drift score increasing for 6+ hours



Every alert must link to:

Relevant dashboard → Filtered logs → Detailed runbook

Alertmanager Rule Example

```
- alert: HighP95Latency
  expr: histogram_quantile(0.95,
    sum by (le) (rate(
      http_request_duration_seconds_bucket{
        job="inference"
      }[5m]))) > 0.3
  for: 10m
  labels:
    severity: page
    team: platform
  annotations:
    summary: "p95 > 300ms"
    runbook: "https://runbooks/latency"
```

Logs & Traces Integration

Unified Troubleshooting Flow



Dashboard Panel

Identifies issue timeframe



Grafana Explore

Pivot to filtered logs



Trace Waterfall

Find bottleneck component

OpenTelemetry Integration

Propagate **traceparent** headers through the full request path:

- API Gateway → Model Serving → Feature Store
- Trace waterfall reveals **which hop is slow**
- Correlate with resource metrics at exact time

Time saved: Cut MTTR from hours to minutes by pinpointing bottlenecks instantly

Release Markers & Experiments



Deployment Context

- CI/CD pipeline writes **annotations** to Grafana
- Vertical markers show exactly when code deployed
- Compare pre/post metrics for canary analysis

A/B Experiment Panels

Win Rate

Success % of variant vs control

Latency Delta

Performance impact of new version

Cost Difference

Resource efficiency comparison



Security & Compliance Widgets



Access Security

Unauthorized attempts, 4xx spikes by IP/CIDR block



Kubernetes Security

Secrets mount violations, privileged pods count



Audit Trail

Create/update/delete events over time



Compliance Status

Policy adherence metrics for SOC2/HIPAA/etc.

Security monitoring provides an additional layer of operational confidence, especially when handling sensitive data or serving regulated industries.

Dashboard Hygiene Rules



Less is More

Maximum **12-16 panels** per dashboard - fight panel creep

Hierarchy Matters

Put **SLO tiles first**, with detailed breakdowns below

Use Proper Units

Always include units (ms, %, W, \$) and consistent time windows

Meaningful Thresholds

Color thresholds (R/A/G) should be set to SLOs, not guesses

Remember: Dashboards are for humans. If it takes more than 5 seconds to understand a panel, redesign it.

Starter Dashboard Templates



01-API-Inference.json

Latency, RPS, errors, HPA activity, region heatmap



02-GPU-Health.json

Utilization, memory, power, temperature, pod mapping



03-Model-Drift.json

PSI/JS divergence, feature missingness, version markers



04-Data-Pipeline.json

Kafka lag, DAG latency, feature staleness tracking



05-Cost.json

GPU hours, \$/1k requests, storage usage, network egress

These templates can be customized with your specific metric names and threshold values to accelerate implementation.

Minimal Metric Instrumentation

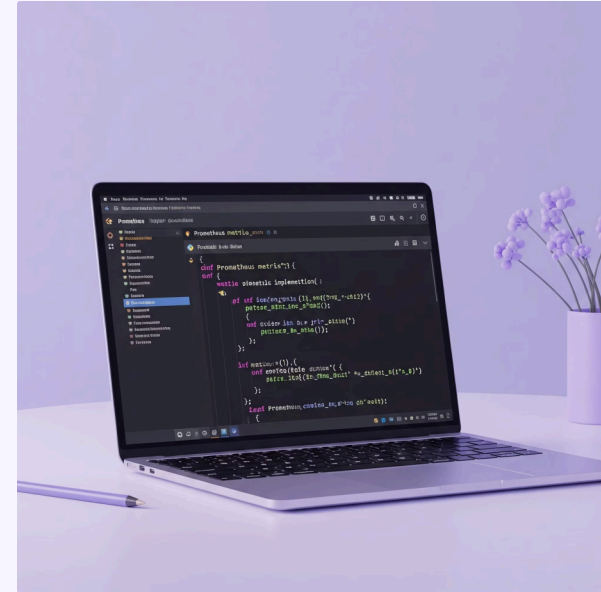
Application Instrumentation Essentials

- Export `/metrics` endpoint for Prometheus scraping
- Emit request duration histogram, in-flight gauge, error counter
- Add **trace IDs** to logs for correlation
- Log **input hashes** (PII-safe) for drift sampling

```
# Python example with Prometheus client
from prometheus_client import Counter, Histogram
```

```
REQUEST_COUNT = Counter(
    'http_requests_total',
    'Total requests',
    ['method', 'endpoint', 'status']
)
```

```
REQUEST_LATENCY = Histogram(
    'http_request_duration_seconds',
    'Request duration in seconds',
    ['method', 'endpoint'],
    buckets=[.005, .01, .025, .05, .1, .25, .5, 1, 2.5, 5, 10]
)
```



OpenTelemetry Trace Example

```
# Add trace context to logs
logger.info("Processing request", extra={
    "trace_id": trace.get_current_span().get_context().trace_id,
    "span_id": trace.get_current_span().get_context().span_id,
    "input_hash": hashlib.sha256(input_text.encode()).hexdigest()
})
```

Tip: Use consistent metric naming patterns across services

Runbooks & Ownership

Essential Runbook Components

Owner

Team responsible for resolving

Diagnosis

Step-by-step troubleshooting

Recovery

Mitigation and rollback steps

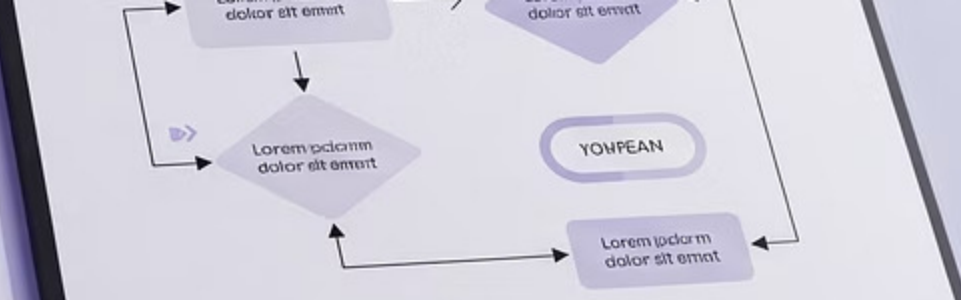
Escalation

Who to contact if steps fail

Runbook Management

- Store in version-controlled repo under `/runbooks`
- Link directly from dashboard panels and alerts
- Monthly incident drills to verify:
 - Alert routing works
 - Contact paths are current
 - Recovery steps are accurate

Reality check: A dashboard without runbooks is just pretty pictures



Key Takeaways



Five Focused Dashboards

Cover API, GPU, Model, Data, and Cost to provide complete visibility



SLO-Driven Design

Connect panels to objectives and actionable alerts with runbooks



Integrated Context

Link metrics, logs, and traces for rapid root-cause analysis



Keep It Lean

Start minimal, iterate as your AI infrastructure scales

Effective monitoring isn't about tracking everything—it's about tracking the right things



Discussion



Which SLO will you enforce first for your AI system?

p95 latency, error rate, or drift score?

Why that one?

Consider the tradeoffs:

- **Latency:** Directly impacts user experience
- **Error rate:** Affects reliability perception
- **Drift score:** Guards against model degradation

Your choice reveals your priorities and constraints.